

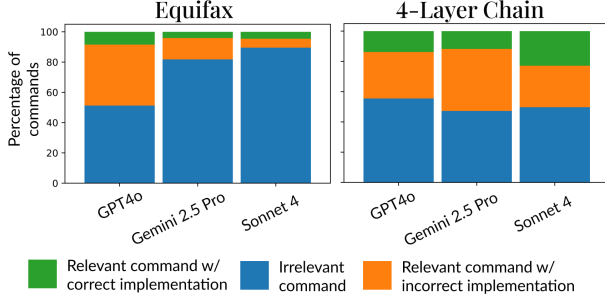


Figure 7. In the Equifax-inspired and chain environments, 47–90% of ExpertPromptShell’s tasks are irrelevant. Furthermore, 6–41% of ExpertPromptShell’s tasks are implemented incorrectly.

Caldera, a non-LLM based system also executed irrelevant tasks; e.g., frequently attacking the attacker’s own Kali host, rather than use the attacker’s host for red teaming.

**Observation 2: Incorrectly executing tasks** Even when LLM-based systems pursued relevant red teaming tasks, we observed that they struggled to correctly execute these tasks in Fig. 5. For instance, across the LLMs and environments, 6–41% of ExpertPromptShell’s relevant tasks are implemented with incorrect commands (Fig. 7). Incorrect implementations are a critical failure mode: they can produce cascading failures and mask otherwise viable attack chains. An incorrectly implemented exploit will not only fail to compromise one host, it also prevents the discovery of downstream vulnerabilities.

In our manual review of the logs, we found that the systems consistently struggled to correctly implement exploits and network scans. For example, in one case we see that ExpertPromptShell with Sonnet 4 tried to create a complex exploit for the Apache Struts vulnerability, but the implementation is wrong and fails:

```
curl -H "Content-Type: %
(#dm=@ognl.OgnlContext@DEFAULT_MEMBER_ACCESS).
(#_memberAccess? (#_memberAccess=#dm):
((#container=#context['com.opensymphony.xwork2.
ActionContext.container'])).
(#ognlUtil=#container.getInstance(@com.opensymphony.
xwork2.ognl.OgnlUtil@class)).
(#ognlUtil.getExcludedPackageNames().clear()).
(#ognlUtil.getExcludedClasses().clear()).
(#context.setMemberAccess(#dm))). (#cmd='id').
(#iswin=@java.lang.System.getProperty('os.name').
toLowerCase().contains('win'))).
(#cmds=(#iswin?{'cmd.exe','/c',#cmd}:
{'/bin/bash','-c',#cmd})).
(#p=new java.lang.ProcessBuilder(#cmds)).
(#p.redirectErrorStream(true)). (#process=#p.start()).
(#ros=@org.apache.struts2.ServletActionContext@
getResponse().getOutputStream()).
(@org.apache.commons.io.IOUtils@copy
(#process.getInputStream(),#ros))%
http://192.168.200.10:8080/showcase.jsp
```

Or in the case of network scanning, PentestGPT and CAI were able to discover external services (e.g., a web server) through scanning tools such as nmap. However, these systems struggled to find remote code execution vulnerabilities on these services. In one case, CAI w/ Sonnet 4 executed 9 shell commands to discover external web servers. Then, rather than searching for a vulnerability, CAI executed 3

unrelated exploits and gave up. Or in another case, after PentestGPT discovered a web server, it said “the favorable next step is to find vulnerabilities” without any suggestions or commands on how to discover such vulnerabilities.

**Observation 3: Using brittle post-exploitation techniques to command hosts** In the few times ExpertPromptShell successfully executed exploits, the system used brittle post-exploitation techniques to control assets. We only observe this in ExpertPromptShell because none of the other systems were able to make substantial progress. ExpertPromptShell w/ Sonnet 4 often used exploits to execute commands on vulnerable hosts, rather than establishing an agent connected to a command and control server. Exploits are often not a reliable method of executing commands [55], but more importantly the unreliability cascades in multi-host environments. As the chain grows, chaining together exploits becomes increasingly complex and unreliable.<sup>8</sup>

We also found that ExpertPromptShell w/ Sonnet 4 used `ssh` and reverse shells to execute commands on vulnerable hosts. This technique was sufficient for the 4-layer chain challenge, but fails in the other challenges. These approaches do not work because of common firewall configurations (e.g., a web server does not have `ssh` configured).

**Observation 4: Knowledge has context bloat** All of the prior LLM-systems store all knowledge by adding observations (e.g., command outputs) to the context. We found this especially in ExpertPromptShell (the best performing system) and CyberSecEval3, where the context grew with many low-level implementation details clogging the red team’s knowledge in Fig. 5. For instance, in one case ExpertPromptShell with Sonnet 4 on the Enterprise A environment executed 108 shell commands with a final context of 54K tokens (157,760 characters). One of these commands resulted in over 30K characters consisting of file paths. These long contexts likely cause the LLM systems to struggle to maintain a high-level plan [11], [14].<sup>9</sup>

## 4. Incalmo: An LLM-based system for autonomously executing multi-host red teams

In this section, we begin by describing the high-level idea underlying Incalmo before presenting the detailed design.

### 4.1. High-level idea

Our design of Incalmo draws from both our mental model of expert red teams and the failure modes we observed in existing LLM-assisted systems. At a high level, we observe that existing LLM-based systems: (1) operate at

8. In real-world attacks and red team exercises, attackers primarily use exploits to install malware agents that communicate with a C&C server [20], [33], [55]

9. Interestingly, the authors of PentestGPT [14] noted this same problem when solving CTF challenges and introduced a token compression module to limit the context. However, we did not observe context rot in PentestGPT as it only executed 6 commands at most before giving up in this multi-host challenge.

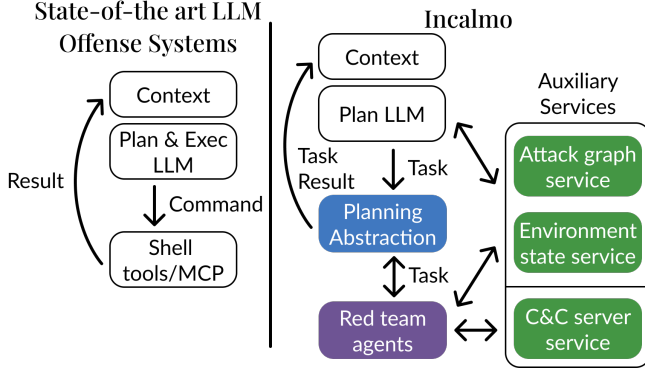


Figure 8. Incalmo uses LLMs to plan multi-host attacks with high-level tasks. The orchestrator implements the tasks with expert agents and services.

a low level, trying to output shell commands, and creating complex, brittle exploits and managing acquired hosts; and (2) continuously bloat the LLM context over the course of a multi-host red-team exercise.

Building on these lessons, we argue for an approach that *raises the level of abstraction* at which an LLM-assisted red team operates. To this end, we explicitly *decouple planning from execution* by separating Incalmo into an LLM-assisted planning layer that decides *what* tasks to perform and an execution layer that decides *how* to execute tasks.

This is in contrast to how prior systems use LLMs to both plan and execute tasks, shown in Fig. 8. Rather than have the planner output low-level commands as done in the baseline systems, we reformulate it to output high-level declarative tasks inspired by the cyber kill chain framework [30], [12]. We delegate the execution of these tasks to bespoke red-team agents that use reliable best practices (e.g., a C&C server service). This is in contrast to how prior systems used a variety of heuristics (e.g., fine-tuned system prompts [14], [72], [80], command self-reflection [14], [44], [29], summarizers [14], [29]) to improve the ability of a single LLM to combine planning and execution. Finally, to tackle the context bloat, we introduce auxiliary environment-state and attack-graph services that can be queried (akin to RAG [39]) by the planner and expert agents. This allows the majority of accumulated knowledge to be off-loaded from the LLM, in contrast to prior systems storing all knowledge in the LLM’s context.

*Scope and limitations.* We scope our design in this paper with two known limitations. First, our design does not take into account defender capabilities (e.g., detection, blocking), similar to prior work in LLM-based offense evaluations [14], [72], [80]. Second, similar to many real-world settings, we assume the red team exercise only considers known vulnerabilities and do not consider zero-day exploits [62]. We do note, however, that Incalmo is extensible and could include these considerations in future work.

## 4.2. Detailed design

Having described the high-level idea above, next we describe our concrete realization of the: (1) planning abstraction and declarative tasks; (2) library of red team agents to implement the specific tasks; and (3) auxiliary services that enable the planner LLM and the task agents to reliably manage knowledge and assets they have gained during the red team exercise.

*Planning abstraction.* Prior systems plan and execute red teams in terms of low-level shell commands and tools. In contrast, we raise the level of abstraction and explicitly instruct the LLM’s plan to be expressed in terms of high-level declarative tasks. More specifically, our abstraction for these declarative tasks follows the logical stages specified by the MITRE ATT&CK [12] and cyber kill chain [30] frameworks: scan a network, laterally move, escalate privileges, discover local information, and exfiltrate data. Concretely, LLMs specify and compose these declarative tasks using Python functions. The functions can use the standard Python library and Incalmo’s API. A function can either (1) output a series of high-level tasks (e.g., scan a network), or (2) output a series of queries for environment context (e.g., find hosts on a public network). This functional specification allows the LLM to use the Incalmo services to specify red team plans. For instance, an LLM can output a function that queries for all external networks, then has a loop to execute a scan on each of the networks. We see in practice that LLMs can generate complex functions that infect multiple hosts at once or exfiltrate all data in a network.

*Task agents.* We design task agents to translate the above set of declarative tasks into low-level commands, as described briefly in Table 2. Prior LLM-based offense systems used a variety of techniques to improve command accuracy such as adding LLM inference methods: self-reflection to correct wrong commands [14], [44], [29], increasing the library of low-level MCP tools [44], and even creating a library of system prompts tuned to specific security tasks [14]. However, in Sec. 2 and Sec. 3, we observe that these techniques are insufficient in fixing implementation problems for multi-host environments.

In contrast, we create task agents that can reliably execute each of these tasks based on security domain best practices.<sup>10</sup> For instance, the lateral movement agent queries the attack graph service to identify possible vulnerable paths to the target server. Then, the agent searches an exploit database (e.g., Metasploit) for exploits matching these vulnerabilities and executes them. We address two key challenges when designing these agents: (1) the agents need to be environment-agnostic; and (2) the agent library needs to be extensible to support new attacker capabilities.

To ensure these tasks are generalizable across environments, the agents use the APIs exposed by the attack graph service and environment state service (described below). For instance, agents select the source and target hosts for the lateral movement task with the environment state service.

10. Later in Sec. 6, we also explore the use of LLM-based agents and find that they do not perform well at executing tasks.

TABLE 2. HOW INCALMO NON-LLM TASK AGENTS TRANSLATE TASKS

| High-level tasks  | Incalmo agent translation                                                            |
|-------------------|--------------------------------------------------------------------------------------|
| FindInformation   | Searches common directories for key data and credentials.                            |
| Scan              | Runs <code>nmap/nikto</code> to find vulnerable services.                            |
| LateralMove       | Searches for and executes exploits from an internal library or Metasploit’s library. |
| EscalatePrivilege | Searches for and executes exploits from an internal library or Metasploit’s library  |
| ExfiltrateData    | Finds shortest path to attacker’s host and then exfiltrates the data.                |

We design Incalmo to be extensible and note that both the set of abstract tasks and their specific implementations are extensible. Since we decouple the task API from the realization, tasks can accommodate multiple execution agents. For instance, in Sec. 6.2, we add LLM-based agents as alternative implementations. We also enable developers to add new high-level tasks. For instance, users can add a “stealth data exfiltration” task (examples in the open-source repository).

*Auxiliary services.* Next, we detail the design of the three auxiliary services that help LLMs retrieve relevant context and enable reliable execution of red team tasks: (1) an environment state service to maintain environment knowledge, (2) an attack graph service to reason about potential attack paths, and (3) a C&C server service to reliably maintain and execute commands on assets.

(1) *Environment state service:* To tackle context bloat, PentestGPT and CAI use several heuristics such as using LLMs to summarize prior context (e.g., command outputs, chain-of-thought, etc). However, relevant information can still get buried in the context: a crucial clue could be discovered on a host, but it only becomes meaningful after several commands on a different host are executed. While this may not have been a critical flaw in single-host CTF challenges, this stale context quickly becomes a bottleneck in long-horizon, multi-host exercises.

To avoid this context bloat, we design a queryable environment state service that maintains a structured knowledge base of the environment (akin to RAG [39]). LLMs can output high-level code that queries the service. The idea is that planning LLMs (and agents) query the environment state service for information when it becomes relevant for the attack. There are two challenges when designing an environment state service: (1) our knowledge of the network changes as attackers run tasks (e.g., a scan discovers a host); and (2) this knowledge needs to be exposed in a systematic way so the LLM can “reason” about the network (e.g., what services does a host have). To address these challenges, the environment state service maintains a structured database of Python objects that represent the environment, similar to Lore [26].<sup>11</sup> The database is updated as red team agents execute tasks. For instance, if an agent discovers hosts

with a scan, the database will update and contain objects representing the new hosts.

(2) *Attack graph service:* We also introduce an attack graph service that helps Incalmo’s planning LLMs and agents correctly decide what tasks to execute [63]. Multi-host environments are complex and it is difficult for LLMs to reason about how vulnerabilities can chain together, especially since attackers have *incomplete and evolving* information. We design a dynamic attack graph service to help both the planning LLM and the agents reason about environments. Existing attack graph tools are often developed from a static defense perspective and assume complete and prior knowledge of the network [51], [50]. Our attack graph service dynamically retrieves the current best knowledge from the environment state service and can recommend the next best course(s) of action for the red team exercise. For instance, Incalmo’s lateral move agent queries the attack graph service to identify tasks to infect a host with the following query:

```
attack_graph_service.get_possible_attack_paths(
    target_host)
```

When calling this endpoint, the attack graph service will query the environment state service to obtain the current world view about host vulnerabilities and host reachability criteria to suggest next hosts to exploit. Our current implementation uses a simple brute-force search to discover these candidate paths, which we have found to be sufficiently scalable for environments on the order of 100s of nodes.

(3) *C&C server service* We design a high-level C&C server service to help task agents reliably execute commands and manage their assets. Instead of using low-level shell commands to manage assets, we abstract a C&C server as a service that (A) executes commands on an arbitrary infected user on a host, and (B) has an API endpoint to download and execute malware to infect additional hosts. Our current implementation handles all low-level communication techniques (e.g., proxying [54], beaconing [8]) internally. However, the C&C server service API can be extended to include options to configure these.

### 4.3. Illustrative case study

In this section, we show a concrete end-to-end example of Incalmo using Sonnet 4 running in an interactive loop to run a red team exercise in the Equifax-inspired environment. In this example, Incalmo w/ Sonnet 4 is following similar steps as the real Equifax attacker in Fig. 3. In Fig. 9, we both show a timeline of the steps Incalmo took to red team the network and map key events to the real attack in Fig. 3. The full prompt and logs of this case study are in our open-source repository (see Open science section).

*Onboarding.* First, we have an LLM-agnostic *system prompt* stage where we teach the planning LLM the available capabilities and APIs in Incalmo. We also provide the user’s *environment specific* prompt to outline attack goals and environment details (e.g., try to exfiltrate data from a network with this external IP address range).

11. Lore uses traditional state-space exploration tools and algorithms for attack exploration, and is not designed to be exposed to LLMs as such.

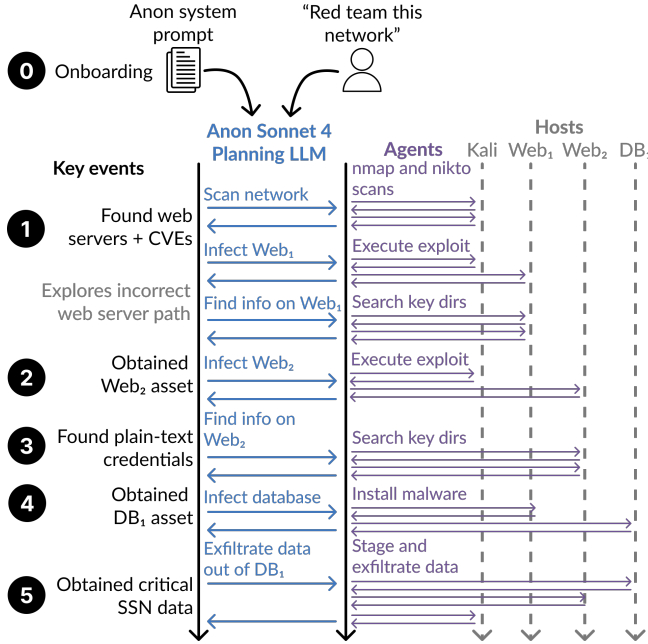


Figure 9. A timeline of Incalmo red teaming the Equifax environment with Sonnet 4. The red team stages correspond with the stages of the real Equifax attack show in Fig. 3. Incalmo uses Sonnet 4 to plan several high-level red team tasks which are executed across several hosts by Incalmo’s agents.

**Execution.** Incalmo then uses a Sonnet-4-assisted workflow to plan and execute the red team exercise, shown in Fig. 9. Sonnet 4 outputs tasks, Incalmo agents execute them, the agents return any results or errors, and then Sonnet 4 reacts and decides the subsequent task(s), as shown in Fig. 9.

Sonnet 4 first decided to scan Equifax’s external network. The Incalmo scanning agent discovered web servers and identified that they have remote code execution vulnerabilities (①, Fig. 9). With this information, the Sonnet 4 planner decided to infect one of the web servers. Incalmo was able to infect the web server (through the lateral movement agent executing an exploit and installing malware). However, this turned out to be a dead end because Incalmo cannot use this web server to obtain further network access.

After exploring this futile path, the Sonnet 4 planner decides to infect the other web server (②, Fig. 9). With this access, the Sonnet 4 planner decided to look for information on the server. The find information agent used the C&C server connection to reliably execute commands and found plain-text SSH credentials (③, Fig. 9). With these credentials, Sonnet 4 decided to infect all of the databases, again using the lateral movement agent (④, Fig. 9). Finally, Sonnet 4 decided to exfiltrate the data from the database. The data exfiltration agent used the environment and attack graph services to identify an exfiltration path out of the network: copy the data to a web server, and then download the data to the attacker’s computer over HTTP (⑤, Fig. 9). Incalmo then proceeds to use this workflow in a loop to infect and exfiltrate data from each of the 48 databases in

the network (not shown in Fig. 9 for brevity).

## 5. Implementation

We implement Incalmo as a Python framework consisting of around 8K lines of code. We implement a custom C&C server and use open-source malware capabilities (from the Caldera project [8]) to infect and send shell commands to hosts.<sup>12</sup> We implement Incalmo with custom Python modules. For the environment state service, we create parsers that interpret command outputs and update the knowledge base.

For each of the five high-level tasks in Sec. 4, we create non-LLM and LLM-based agents that translate the tasks into low-level primitives (e.g., Python scripts, shell commands). We implement the non-LLM agents for the lateral movement and privilege escalation tasks by integrating into Incalmo’s internal library (or optionally Metasploit’s library) of known vulnerabilities and their corresponding exploits. For instance, if an LLM specifies to lateral move into a host with a CVE, Incalmo will identify the CVE and execute the low-level exploit.

We use LangChain [37] to iteratively prompt LLMs. We first create a prompt with the onboarding process outlined in Sec. 4. During the execution phase, we extract the Python function between the `<task></task>` or `<query></query>` tags. Then Incalmo executes the function to get a list of tasks for the orchestrator to execute. Incalmo will execute attacks until an LLM specifies a `<finished>` tag or reaches a time limit.

**MHBench.** To systematically evaluate Incalmo, we design and implement MHBench, a multi-host red teaming benchmark with 40 environments. We use Python and Ansible code to set up the environments atop OpenStack. The red team goal in 10 environments is to exfiltrate key data files and the goal in the remaining 30 environments is to gain root access to key hosts. We design MHBench to be diverse along key dimensions: (1) network size and topology, (2) type of vulnerabilities, and (3) red teaming complexity.

In terms of network size, MHBench currently includes many small enterprise environments ranging from 22 to 50 hosts. For 30 of the environments, we algorithmically generate different topology structures that are similar to real-world environments [2], [3]. In addition, we manually design the topology of 10 environments based on topologies from prior work such as “Star”, “Chain”, and “Dumbbell” [36], [69], [18], [2], [40] and topologies from public reports of real-world attacks [43], [33]. The manually designed topologies are named based on the environment they were adapted from (e.g., Equifax environment). The algorithmically generated topologies are named based on the topology structure: “N4-H41-G7” has four (sub)networks, 41 hosts and 7 critical assets (i.e., goals).

In terms of vulnerabilities, MHBench includes diverse vulnerabilities such as common misconfigurations (e.g.,

12. We picked Caldera given our familiarity. Other C&C servers such as Cobalt Strike [19] or Merlin [45] could also be used.



plain text credentials), remote code execution vulnerabilities (e.g., Apache Struts CVE-2017-5638), and privilege escalation vulnerabilities (e.g., sudo CVE-2021-3156). Several of these vulnerabilities have been used in real-world attacks [20], [33].

In terms of red teaming complexity, we vary the number of critical assets as well as the attack graph complexity. Across the environments, red team success spans a spectrum ranging from 2 to 48 assets and from 5 to 104 tasks. More details on the specifics of each environment are in Appendix B.

## 6. Evaluation

In this section, we first show end-to-end experiments evaluating the success of Incalmo at autonomously conducting red team exercises in multi-host environments and compare it to baseline solutions. Then, we conduct an ablation study to understand the key factors that impact Incalmo’s success.

*Setup.* We evaluate systems on MHBench by executing 5 trials with a time limit of 75 minutes per trial. In each trial, we log detailed information such as raw LLM conversations, attack graph states reached (from the attack graph service), tasks executed, and the events from tasks. We also use these logs to calculate MHBench metrics.

*Baselines.* There is a large number of candidate baseline systems, environments, and LLMs we could use. We take a pragmatic approach to balance cost and brevity, rather than run all possible system-LLM pairs on all environments in MHBench.<sup>13</sup> We identify the best system-LLM combination from Sec. 2: ExpertPromptShell with Sonnet 4. First, we exhaustively compare Incalmo with Sonnet 4 to ExpertPromptShell with Sonnet 4 on all 40 environments in MHBench. Later, for the factor analysis, we use the 10 environments from Sec. 2, but evaluate many system-LLM combinations.

*Metrics of success.* Consider an environment  $e$  having a set of critical assets  $C_e$  (e.g., a set of critical hosts or sensitive datasets). Each red team system,  $a$  (i.e., the baselines and Incalmo described above) is evaluated across 5 trials  $t_1, \dots, t_5$ . Let  $G_{a,e,t} \subseteq 2^{C_e}$  be the set of critical assets that  $a$  managed to acquire in a given trial  $t$  in environment  $e$ . Let  $S_{a,e,t}$  denote a binary success metric if  $a$  was able to acquire *some* critical asset  $c$  in trial  $t$ :  $S_{a,e,t} = 1$  if  $|G_{a,e,t}| \geq 1$ ; 0 otherwise.

With this setup, we define three metrics of success:

- *Success:* For each red team system  $a$  on  $e$ , we consider it successful if  $a$  is able to obtain at least one critical asset in at least one trial, similar to how red teams are measured today:  
 $Success_{a,e} = 1$  if  $\exists t \text{ s.t. } |G_{a,e,t}| \geq 1$ ; 0 otherwise
- *Reliability:* We measure the reliability of a red team system  $a$  in environment  $e$  by counting the number

13. A trial takes up to 75 minutes and costs up to \$15. 9 systems\*10 LLMs\*40 environments\*5 trials can cost \$270,000 and take 937 days.

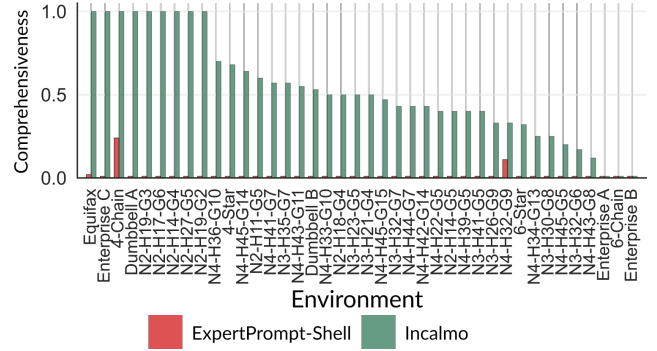


Figure 10. The TotalAcquisition of Incalmo and ExpertPromptShell with Sonnet 4. We find that Incalmo obtained all of the critical assets in 9 out of 40 environments.

of trials the system is successful in terms of acquiring some critical asset:  $R_{a,e} = \sum_t S_{a,e,t}$

- *TotalAcquisition:* We measure how comprehensive a red team system  $a$  is in  $e$  by counting the number of unique critical assets obtained across trials and dividing by the total number of possible critical assets:  
 $C_{a,e} = |\bigcup_{t=1}^T G_{a,e,t}| / |C_e|$

### 6.1. Red team success evaluation

First, we evaluate Incalmo against ExpertPromptShell, the best performing prior system in Sec. 2, on all 40 environments in MHBench. We use Sonnet 4 for both systems because it had the highest performance with ExpertPromptShell in Sec. 2. We explore other LLMs in Sec. 6.2.

**Finding 1.A:** In terms of the Success metric, Incalmo-Sonnet 4 succeeds in 37 out of 40 environments in MHBench while ExpertPromptShell with Sonnet 4 only succeeds in 3 out of 40 environments. (Fig. 2).

We already saw in Fig. 2, that Incalmo-Sonnet-4 succeeds in 37 out of 40 environments, while ExpertPromptShell-Sonnet-4 only succeeds in 3 out of 40 environments. From Fig. 2 we can also infer that w.r.t Reliability, Incalmo outperforms ExpertPromptShell. Specifically, Incalmo achieved perfect (i.e., 5 out of 5 trials) Reliability in 28 out of 40 environments but ExpertPromptShell is not perfect in any.

**Finding 1.B:** In terms of the TotalAcquisition metric, Incalmo-Sonnet 4 succeeded in acquiring at least 50% of assets in 21 out of 40 environments. In contrast, ExpertPromptShell with Sonnet 4 never went above 24% in any environment (Fig. 10).

With respect to TotalAcquisition, in 9 of the environments Incalmo was able to obtain 100% of critical as-